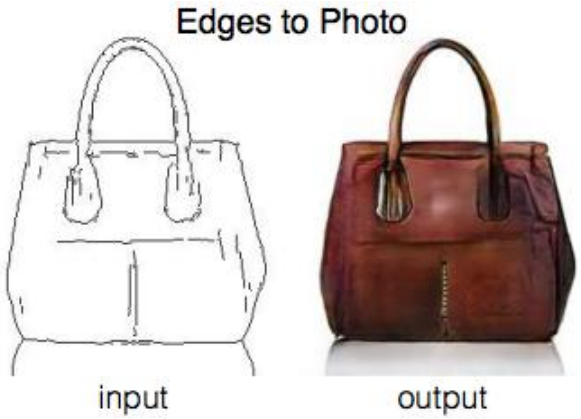
방법론 세미나

이지훈

GAN 모델

- 생성적 적대 신경망 , Generative Adversarial Network
- 생성모델(generative model) 딥러닝
- 생성자와 식별자라는 신경망 2개가 서로 경쟁하면서 학습
- 주요 목적: 이미지 생성
- 넓은 적용 범위로 예시와 같이 일반 말에서 얼룩말을 생성하거나 스케치로 부터 실제 사진을 생성하는 등 여러 방면으로 사용되고 있다.



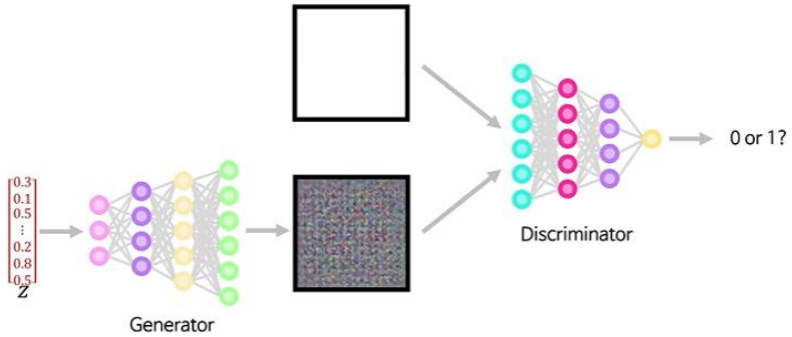
생성적 적대 신경망

- 생성적 적대 신경망(GAN)은 두 네트워크가 서로 경쟁을 하는 구조이기에 적대적이라는 의미가 이름에 포함된 심층 신경망의 일종이다.

  *생성자(Generator) 신경망 G는 인공적인 객체(가짜 데이터)를 생성.

  *식별자(Discriminator) 신경망 D는 이미지가 진짜인지 가짜인지를 식별.

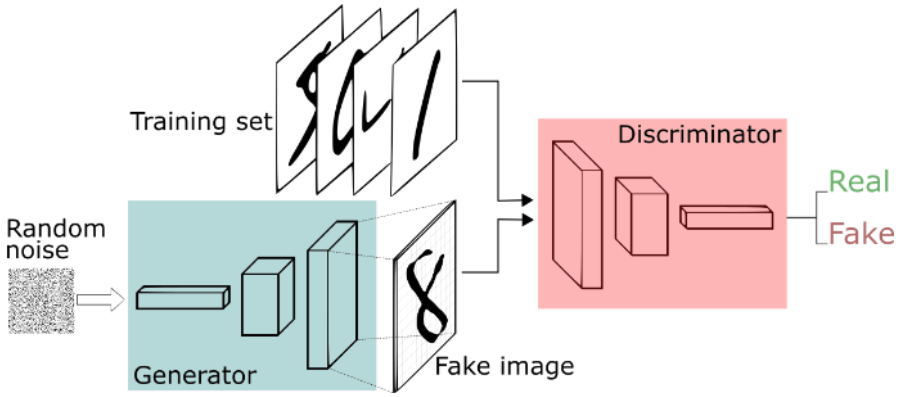
생성자는 식별자가 속을만한 가짜 그림을 그리려고 노력하고 식별자는 가짜 그림임을 알아내려고 노력하는 과정에서 점점 원본과 비슷한 그림을 그린다. (생성자 G를 훈련시킨다)



생성적 적대 신경망

- 처음에 생성자 G는 랜덤 노이즈만 발생시킨다, 개념적으로 식별자 D가 생성할 이미지에 대한 지침을 생성자 G에 제공하게 된다.
- GAN은 실제 이미지와 생성된 이미지를 학습함으로써, 어떤 특징들이 이미지가 실제인지를 감별하는데 사용되는지를 식별자 D가 학습되도록 한다.
- 그 이후 동일한 식별자 D가 생성자 G에 피드백을 제공하여 실제 이미지처럼 보이는 그림을 만들도록 한다.

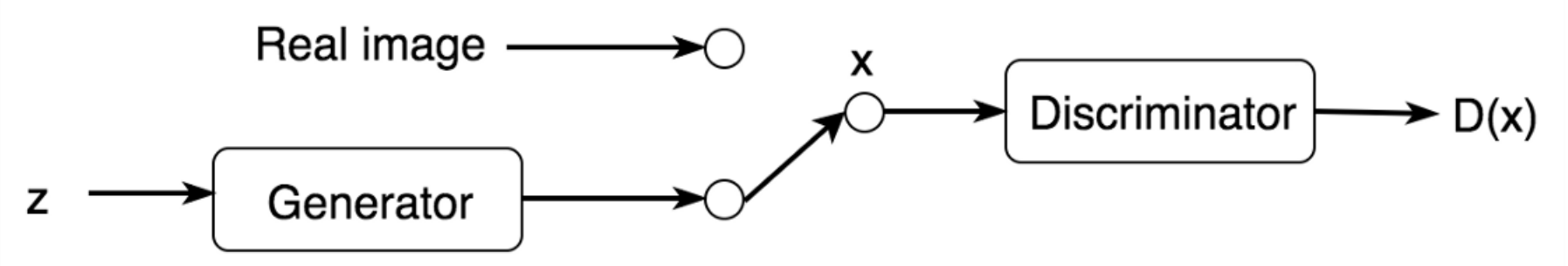
(에포크가 커질수록 원본과 비슷한 이미지가 생성된다)



훈련

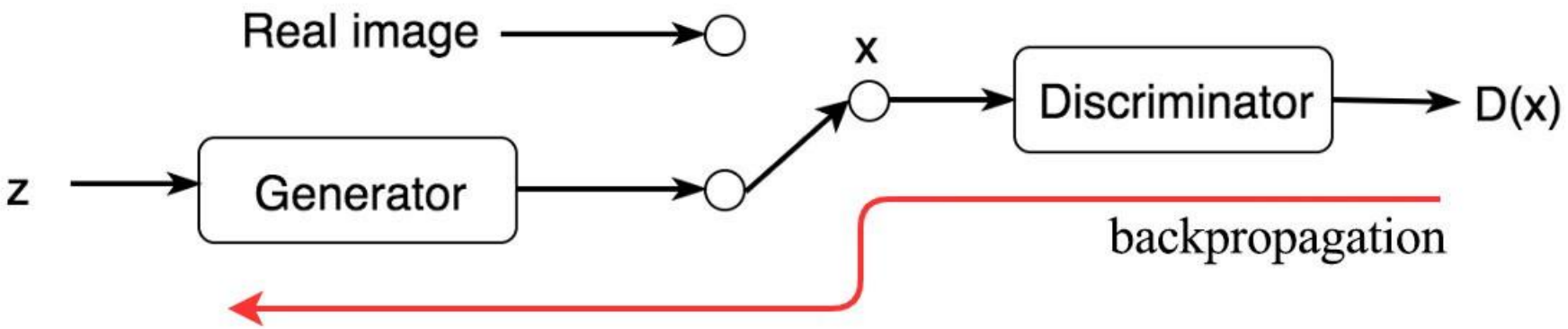
- 식별자는 실제 이미지와 생성된 이미지를 입력으로 사용하여 실제인지/생성된것 인지를 구별.
- 입력이 진짜라면 D(x)=1이 되도록, 가짜(생성된 것)이라면 D(x)=0이 되도록 식별자를 훈련
- 이 프로세스를 통해 식별자 D는 진짜 이미지 식별에 기여하는 특징들을 학습하게 됨

(D(x)의 출력은 입력 x가 실제일 확률)



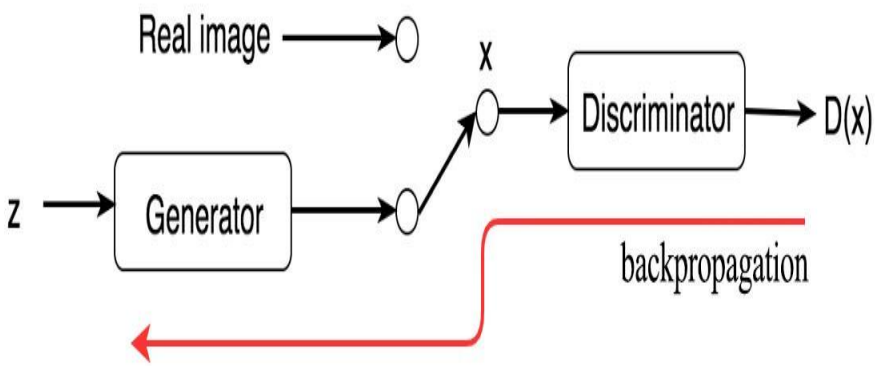
훈련

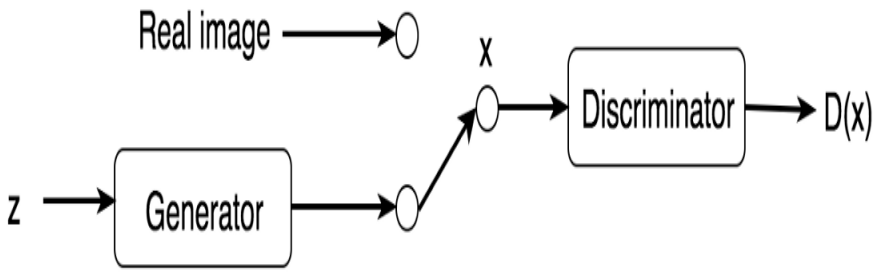
- 생성자는 D(x)=1 (실제 이미지)로 이미지를 생성하길 원한다.
- 따라서 훈련된 식별자를 고정하고, 이 목표 값을 다시 생성자로 역전파하여 생성자를 훈련
- 생성자가 식별자가 진짜로 생각하는 것 과 같은 이미지를 생성하도록 훈련



훈련

- 두 네트워크(식별자/생성자)를 번갈아 가며 훈련시키고, 각각을 향상시키기 위해 경쟁에 빠지게 한다, 결국 식별자는 실제와 생성된 것 사이의 작은 차이를 식별하고, 생성자는 식별자가 차이를 말할 수 없는 이미지를 작성하여 점점 자연스러운 이미지를 생성

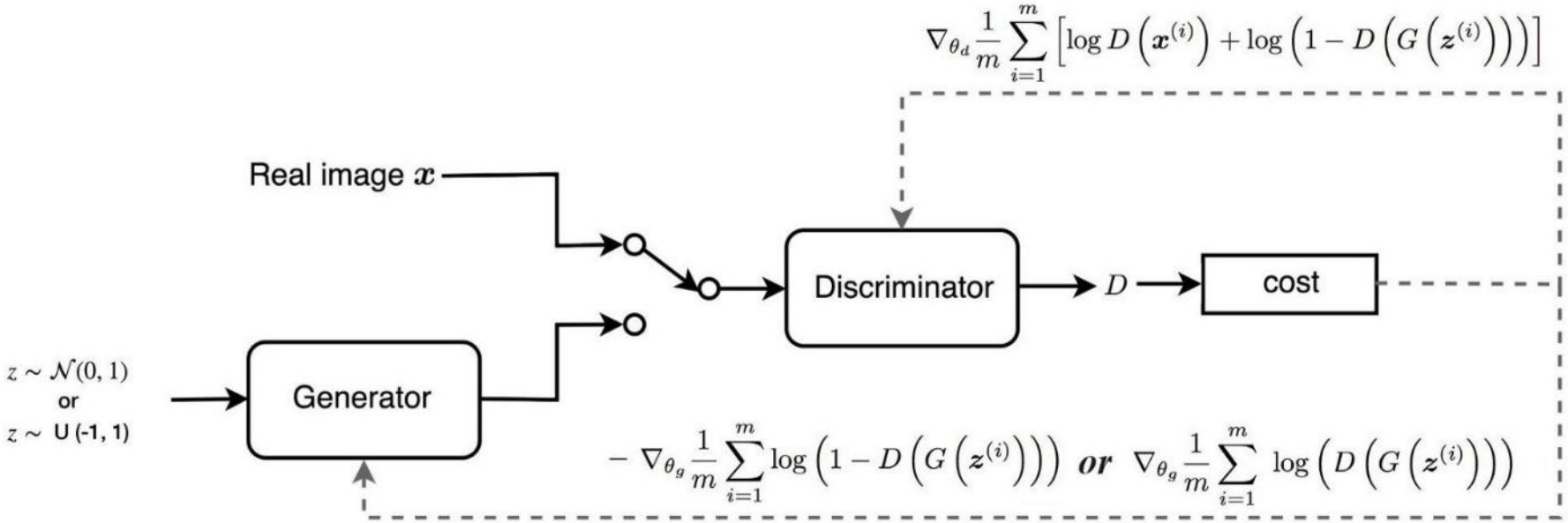




역전파

- 실제 이미지를 실제로, 생성된 이미지를 가짜로 인식할 수 있는 확률을 높임

  *진짜 판별 D(x)는 1에 가깝게 , 가짜 판별 D(G(z))는 0에 가깝게



GAN

실습

```
[1] %tensorflow_version 2.x
    import tensorflow as tf
    !pip install imageio    # GIF를 만들기위해 설치합니다.
    import glob
    import imageio
    import matplotlib.pyplot as plt
    import numpy as np
    import os
    import PIL
    from tensorflow.keras import layers
    import time

    from IPython import display
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
Requirement already satisfied: imageio in /usr/local/lib/python3.11/dist-packages (2.37.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from imageio) (2.0.2)
Requirement already satisfied: pillow>=8.3.2 in /usr/local/lib/python3.11/dist-packages (from imageio) (11.1.0)
```

```
mnist = tf.keras.datasets.mnist
(train_images, train_labels), (_, _) = mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5   # 이미지를 [-1, 1]로 정규화합니다.

BUFFER_SIZE = 60000
BATCH_SIZE = 256
# 데이터 배치를 만들고 섞습니다.
train_dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

https://colab.research.google.com/drive/1s-aF82CLv5etSE6J2q2gUFRNUfSQZEsf?usp=sharing

- 생성자와 감별자를 훈련하기 위해 MNIST데이터 셋 사용, 생성자는 손글씨 숫자 데이터를 닮은 숫자들을 생성

- 사이즈 28*28*1의 이미지가 나오도록 업샘플링

```
def make_generator_model():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))
    assert model.output_shape == (None, 7, 7, 256) # 주목: 배치사이즈로 None이 주어집니다.

    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding='same', use_bias=False))
    assert model.output_shape == (None, 7, 7, 128)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding='same', use_bias=False))
    assert model.output_shape == (None, 14, 14, 64)
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding='same', use_bias=False, activation='tanh'))
    assert model.output_shape == (None, 28, 28, 1)

    return model
```

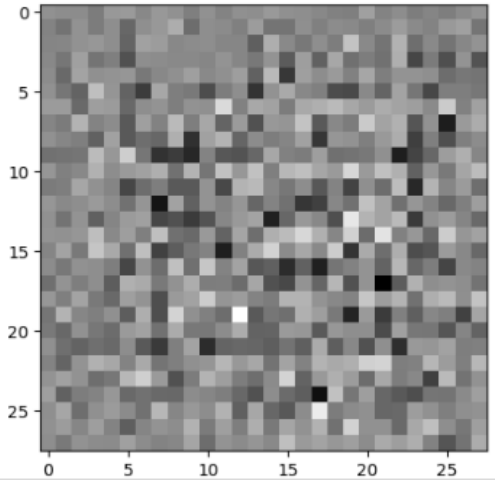
GAN

```
generator = make_generator_model()

noise = tf.random.normal([1, 100])
generated_image = generator(noise, training=False)

plt.imshow(generated_image[0, :, :, 0], cmap='gray')
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
<matplotlib.image.AxesImage at 0x7b7adcd7e290>
```



실습

- 아직 훈련이 되지 않은 생성자를 이용해 이미지를 생성

GAN

```
[5] def make_discriminator_model():
        model = tf.keras.Sequential()
        model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same', input_shape=[28, 28, 1]))
        model.add(layers.LeakyReLU())
        model.add(layers.Dropout(0.3))

        model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same'))
        model.add(layers.LeakyReLU())
        model.add(layers.Dropout(0.3))

        model.add(layers.Flatten())
        model.add(layers.Dense(1))
        return model
```

```
[6] discriminator = make_discriminator_model()
    decision = discriminator(generated_image)
    print (decision)
```

```
tf.Tensor([[-0.00074899]], shape=(1, 1), dtype=float32)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[7] # 이 메서드는 크로스 엔트로피 손실함수 (cross entropy loss)를 계산하기 위해 헬퍼 (helper) 함수를 반환합니다.
    cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
```

```
[8] def discriminator_loss(real_output, fake_output):
        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        total_loss = real_loss + fake_loss
        return total_loss
```

```
[9] def generator_loss(fake_output):
        return cross_entropy(tf.ones_like(fake_output), fake_output)
```

```
[10] generator_optimizer = tf.keras.optimizers.Adam(1e-4)
     discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

실습

- 식별자를 사용하여 생성된 이미지가 진짜인지 가짜인지 판별 (진짜 = 양수의 값, 가짜 = 음수의 값) 후 출력
- 진짜 이미지에 대한 식별자의 예측과 1로 이루어진 행렬을 비교하고, 가짜 이미지에 대한 식별자의 예측과 0으로 이루어진 행렬을 비교한다.

실습

```
[12] EPOCHS = 50
     noise_dim = 100
     num_examples_to_generate = 16

     seed = tf.random.normal([num_examples_to_generate, noise_dim])
```

- 훈련 루프 정의
- 훈련 루프는 생성자가 입력으로 랜덤시드를 받는 것으로 시작해 그 시드값을 사용하여 이미지를 생성.

GAN

```
checkpoint.restore(tf.train.latest_checkpoint(checkpoint_dir))
```

```
# 에포크 숫자를 사용하여 하나의 이미지를 보여줍니다.
def display_image(epoch_no):
  return PIL.Image.open('image_at_epoch_{:04d}.png'.format(epoch_no))

display_image(EPOCHS)
```

```
anim_file = 'dcgan.gif'

with imageio.get_writer(anim_file, mode='I') as writer:
  filenames = glob.glob('image*.png')
  filenames = sorted(filenames)
  last = -1
  for i,filename in enumerate(filenames):
    frame = 2*(i**0.5)
    if round(frame) > round(last):
      last = frame
    else:
      continue
    image = imageio.imread(filename)
    writer.append_data(image)
  image = imageio.imread(filename)
  writer.append_data(image)

import IPython
if IPython.version_info > (6,2,0,''):
  display.Image(filename=anim_file)
```

```
try:
  from google.colab import files
except ImportError:
  pass
else:
  files.download(anim_file)
```

실습

- 모델 훈련
- Train() 메서드를 생성자와 식별자를 동시에 훈련하기 위해 호출.

점점 숫자의 모습으로 보여진다

- 훈련 중 저장된 이미지를 사용해 GIF를 만듬

결론

- 생성적 적대 신경망 GAN은 생성자와 식별자가 서로 경쟁하도록 학습하여 학습과정에서 점차 원본 이미지와 비슷해지는 가짜 이미지를 생성해내게 한다.

장.단점

- GAN은 다양한 데이터를 대상으로 새로운 데이터를 생성할 수 있어 활용 범위가 넓지만, 훈련의 불안정성 이라는 단점을 가진다.

활용 방안

- 데이터 보강, 텍스트-이미지 변환, 이미지/비디오 생성/개선, 선호도 분석, 데이터 품질 개선